Night Watchman Project

Document: Night Watchman Project
Version: v1.0.0
Author: Celaya Solutions
Contact: hello@celayasolutions.com
Date: 2026-09-06
SHA256: 5393d528299f8d5230f119a7f14b21334763926fb05fadf2ec3ac6bdb626e113
Chain: n/a
Tx: [not anchored]

The maintained Level 2 implementation lives here. Start with the learner lesson and preparation. The legacy starter folder contains compatibility pointers and a generated workflow, not another editable implementation.

Commands from the course root

Command	Result	External effects
`uv run --frozen zta doctor watchman`	Checks the supplied project files	None; does not validate hosted access
`uv run --frozen zta test watchman`	Deterministic temporary-file/API-fixture tests	No network, issue, or paid API
`uv run --frozen zta prepare watchman`	Installs workflow and private worksheet; keeps existing files	Local file copies only
`uv run --frozen zta start watchman`	One local rehearsal; exits after printing receipt	No network; local state/alerts under `.zta/watchman/`
`python projects/watchman/watch.py --github`	One hosted round	Only from the deliberately enabled Actions workflow in a learner fork

No model download, model key, personal GitHub token, always-on server, database service, or Railway deployment is required. The hosted job uses Python's standard library; it does not install the document app or send files to a model.

Files and authority

File	Purpose
`watch.py`	Validates one signal, coordinates state and one practice issue, writes receipts
`manage.py`	Local preparation and file checks
`workflow.template.yml`	Canonical workflow template; install in learner fork only
`tests/test_watchman.py`	Behavior tests including write/fetch/issue failures and recovery
`../../courses/project-lab/level-02/assets/practice-page.html`	One shared synthetic practice input; OPEN or PAUSED
`.github/workflows/watchman.yml` at repo root	Installed workflow on learner default branch
`.watch-state/watchman.json` at repo root	Hosted public state saved with GitHub Contents API
`.zta/watchman-run/rounds.jsonl`	Per-run safe receipt, uploaded alone as a 30-day artifact
`.zta/PROJECT-LAB-02.md`	Private learner predictions and evidence; ignored by Git

The hosted source URL is constructed from the current repository, the workflow run's immutable commit SHA, and the fixed practice-page path. The stable source identity in state uses that repository's default branch. No arbitrary URL input is accepted. Redirects and proxy inheritance are refused; fetches have a 15-second timeout and 128 KiB page cap. API responses are limited to 2 MiB. The job has a five-minute timeout. There are no automatic network retries.

The job accepts only schedule/manual triggers on the default branch, requires WATCHMAN_ENABLED=true and the temporary token, rejects the canonical course repository, and confirms a public repository with Issues enabled. The concurrency group serializes this installed workflow and does not cancel a running job. A pending queued job may be superseded by GitHub; learners should start one run at a time and inspect it. GitHub concurrency

The job token has only contents/issues write permissions; unspecified permissions are none. These are repository-wide scopes. The code's writes are the one state file and practice issues in the same fork. It does not add comments, delete content, follow page instructions, trigger arbitrary commands, or post to another service. Checkout does not persist credentials; the token is passed only to the watcher step. Only the exact receipt file is uploaded, never `.zta/` as a directory.

State and failure sequence

The state schema contains source, current value, sequence number, and optional pending transition. A baseline is saved before reporting success. Unchanged rounds leave state alone; dated receipts are kept by Actions. An invalid or unreadable state fails; it is never treated as an empty baseline. State updates carry the previously read file SHA so a conflicting write fails instead of silently overwriting another change. GitHub Contents API

For a change:

1. Persist the pending transition (old/new values plus an ID derived from source and next sequence).

2. Look for that ID in this fork's open and closed issues created by `github-actions[bot]`. Search is bounded to ten pages of 100 results. A search limit or API failure stops the round.

3. Persist `attempted: true` before sending. A process crash after this point cannot cause an automatic resend.

4. Create the one practice issue if no prior attempt was made. A definitive HTTP rejection may mark the pending attempt retryable; repair the specific block first.

5. Persist the confirmed new value and clear pending state only after an issue receipt is known. A failed final save leaves pending state for reconciliation.

If a reply is lost after GitHub created the issue, the next round finds its exact marker and completes state without creating another. If delivery remains uncertain and no issue is found, the watcher fails and requires human review. This favors avoiding duplicate alerts over automatic recovery from every outage; it does not claim a distributed exactly-once guarantee. GitHub Issues API

A recovery round completes pending work before reading a fresh page. Its receipt says `alert recovered`; a following new round makes the next observation. Every valid status transition receives a new sequence, so OPEN-to-PAUSED can alert again after a separate PAUSED-to-OPEN transition.

Stop and recovery runbook

Disable **Project Lab Watchman**, set WATCHMAN_ENABLED=false, and cancel queued/active work. Wait for final run states before editing a workflow or diagnosing state. A disabled trigger cannot undo a request already sent. Keep the last good state and any existing issue.

- Fetch/signal error: fix the supplied input or wait for service recovery, then deliberately start one new run after enabling.

- State write denied: inspect branch protection/rules and the job's scopes. Use a dedicated learner fork; do not weaken an employer repository.

- Definitive issue denial: enable Issues or fix the policy that caused the specific rejection; the pending transition can retry.

- Issue exists but state is pending: compare the marker, old/new values, and source. A new run should find the bot-authored issue and complete state.

- Uncertain delivery, no matching issue: keep disabled. Review the pending ID and all relevant open/closed issues, including earlier pages. If the result cannot be established, stop and retain the failed evidence. Do not clear `attempted`, delete state, or keep retrying to force a pass. A repository owner must explicitly review any manual resolution; document it as a repair, not a successful original run.

Download useful receipts before their artifact expiry. Failures before Python starts appear only in the GitHub job log; abrupt job cancellation may also prevent a final application receipt. A receipt reports a completed check or error, not proof of an unobserved schedule tick.

Maintainer checks

Run both project test commands, the course validators, and the builds in release maintenance. `scripts/sync_watchman_starter.py` refreshes only the legacy workflow copy. The two small legacy Python entry points route to this project from a complete course clone and fail with guidance when downloaded alone.

The workflow remains a template in canonical source. Hosted tests require a specifically approved practice fork, intentional enabling, and shutdown afterward. See Level 2 verification. Do not infer a Windows device run, GitHub issue, schedule firing, or student upload from local fixtures.